

**Design and Implementation of an Affordable, PWA-Enabled
Digital Operations Web Application for Workforce Scheduling
and Management in Small Businesses**

Eco-Clean

Final Report

Vidarshan Rathnayake | Upul Atapattu

Course: CSIS 4495	Section: 002
Full Name	Student Number
Atapattu, Upul M [Team Lead]	300392188
Arachchi Mudiyanseelage, Vidarshan	300392818

Table of Contents

1.	Introduction.....	1
1.1.	Research Question & Hypothesis	1
1.2.	Research Methodology	1
2.	Evolution of the Project.....	3
2.1.	Originally Proposed Work Scope	3
2.2.	Changes to the Proposed Work Scope based on Industry Partner Feedback	3
2.3.	Expansion of Scope	4
3.	System Architecture and Design.....	5
3.1.	Technology Stack	5
3.2.	Database Design.....	6
3.3.	Eco-Clean Web Codebase Structure	10
3.3.1.	High-Level View	10
3.3.2.	Folder Structure	11
3.3.3.	Frontend Structure	12
3.3.4.	Backend Structure.....	14
3.3.5.	Data Access Layer.....	15
3.3.6.	Core Shared Library (^lib/^).....	15
3.3.7.	AI Architecture	16
3.3.8.	Database and Persistence Layer.....	16
4.	Key Features and Functionalities.....	18
5.	Evaluation Techniques	33
5.1.	Functional and Workflow Testing.....	33
5.2.	Payroll Validation	33
5.3.	User Feedback and Client Evaluation.....	33
5.4.	Usability and Interface Evaluation.....	34
5.5.	Deployment and Environment Validation	34
6.	Project Planning & Timeline	34
7.	Challenges Encountered	37
8.	Conclusion.....	38
9.	Future Improvements.....	38
10.	External Partners & Affiliates	39
11.	Work Logs	40
12.	AI Usage	43
13.	References	46

1. Introduction

Eco-Clean is a digital scheduling platform designed for small service businesses such as cleaning companies, which improves appointment management, staff coordination, availability tracking, leave handling, and timesheet workflows. Its novelty lies in combining a centralized web platform with a staff-focused Progressive Web Application (PWA) for delivering a practical, low-cost solution.

As the current workflow relies heavily on spreadsheets, emails, and manual coordination, leading the company to inefficiencies and scalability constraints. This applied research project focuses on designing and implementing a scalable digital platform that integrates scheduling, staff allocation, appointment tracking, and managing the workforce, including payroll management. Furthermore, this project also added AI features on top of the core features, augmenting the value.

1.1. Research Question & Hypothesis

Research Question that we are trying to answer is, “How a digitally integrated scheduling system contribute to the improvements in operational efficiency and reduce administrative overhead for a service-based cleaning business and how can we design and implement such a platform?”. Therefore, the Hypothesis is as follows:

“A centralized scheduling system supported by a web application and Progressive Web Application (PWA) will reduce manual coordination, improve visibility of staff availability and appointments, and provide a cost-effective digital workflow for small service operations.”

1.2. Research Methodology

The project took an applied research approach, where the requirement gathering and system design decisions were directly informed by industry partner(Nettoyage Eco Vert) input, iterative validation and real-world observations. Instead of relying solely on assumptions, the following research methods were used to understand operational challenges and translate them into practical system features.

Stakeholder Interviews

Multiple discussions were conducted with the industry partner to understand the current operational workflow and expectations from the system, which helped identify key pain points of a fragmented scheduling process, and inefficiencies in coordinating staff and appointments. The insights gathered from these interviews influenced the prioritization and the shape of the features such as centralized scheduling and integrated payroll workflows for administration work, and staff accessible interface for day to do operations.

Operational Shadowing

Complementing the mentioned interviews, day-to-day administrative activity processes were observed, including staff scheduling and payroll functions, which provided a clearer understanding of how tasks were performed in practice, beyond what was described conceptually. Through this observation, several inefficiencies were identified, such as repetitive data entry, manual tracking of staff availability, and dependency on informal communication methods. These findings reinforced the need for automation and real-time system interaction through a staff-facing interface.

Existing Platform Analysis

A review of existing scheduling and workforce management solution (Connecteam) was conducted to understand limitations to handle the current workflow. While the platform offered structured scheduling capabilities, the appointment management and scheduling capabilities were not offered in the same platform, which was not tailored to the operational needs of small service-based businesses. Furthermore, GetJobber.com, the company's previous system prior to Connecteam, is deemed to be expensive although it offered a suitable appointment management and scheduling platform. This analysis highlighted a gap for a lightweight, integrated solution that balances functionality with simplicity and affordability.

Technology Exploration

Modern frameworks and technologies were evaluated based on scalability, development efficiency, and deployment feasibility. Special attention was given to solutions that minimize operational costs while maintaining performance and usability. This led to key decisions such as adopting a full-stack Next.js architecture and implementing a Progressive Web Application (PWA) instead of native mobile applications.

Design Translation

The findings from interviews, observations, and technical analysis were consolidated into system design decisions. Each feature was mapped to a real operational need, ensuring that the system remains aligned with practical business requirements. This approach ensured that the final implementation was not only technically sound but also directly applicable to real-world usage scenarios.

2. Evolution of the Project

2.1. Originally Proposed Work Scope

Web-Based Scheduling Platform

- User-Friendly Interface to accommodate the needs of client, front line worker and administrative staff
- Scheduling management system for booking, rescheduling, and cancelling appointments.
- Admin & Staff Dashboard for monitoring and managing schedules and assigning resources
- Google Maps integration for service area validation and route planning.
- Calendar Integration for both client and staff regarding the appointments
- Notifications and Reminders on bookings and updates

Mobile Application (iOS & Android)

- Client interface to view, book, and manage appointments and receive real-time updates
- Staff interface to access daily schedules, view client addresses and notes, and navigate via integrated maps. Additionally, if the frontline staff is delayed, they can update the client on the delay.

2.2. Changes to the Proposed Work Scope based on Industry Partner Feedback

After having a detailed discussion with Stephanie Riddelle, the founder of Nettoyage Eco Vert, we have the following changes to the original proposal.

- Even though the original proposal initially talked about client-side mobile interface, Stephanie has said the business is not driven by the client-side online booking. Therefore, she did not want to direct our efforts on this.
- The business runs NOT on shift basis and staff travel from their houses to the appointment places.
- Staff Time Tracking is a Must Have, with GPS-based clocked in/out. The interface has should have a count-down timer for the remaining appointment time. The appointments should have 4 different sections for the Location information, WiFi details, Other instructions and Attaching photos.
- Staff interface should have a way to update their availability, apply leave 2 weeks prior. The staff app should be connected to Google Maps for navigation.
- The web application should connect with Google Maps to calculate travel time. Also, when scheduled, the calendar should offer a repeat option.
- Most preferred design idea can be inspired from <https://www.getjobber.com/>

The project underwent several key transformations from the initial proposal to the final implementation.

Shift to PWA Architecture

Originally, the system included a native mobile application. However, this was replaced with a **Progressive Web Application (PWA)** to reduce development and deployment costs while maintaining mobile usability. This decision was taken after discussing with the industry partner, as it brings the following benefits:

- It eliminated app store dependency and related costs.
- Reduced development overhead
- Enabled faster deployment and updates
- Still provided mobile-friendly staff interaction

PWA is a web-based application that uses browser capabilities in order to deliver app-like experience to users. Compared to traditional mobile applications, a PWA does not require installation through Apple App Store or Google Play Store. These PWAs are built using standard web technologies (HTML, CSS, JavaScript) but also include additional capabilities such as:

- Offline functionality
- Home screen installation
- Push notifications
- Background synchronization
- App-like full-screen experience
- Responsive design for mobile

Since our current web-app is being built using Next.js (App Router), the transition to a PWA model was viable with minimal architectural change. It further has given us chance to add additional work to evolve the system into a full operations platform.

2.3. Expansion of Scope

In order to turn our solution in to a end to end operational platform, we have incorporated the following:

- Staff Profile Management
- Availability and leave tracking
- Time tracking via work sessions

- Payroll and pay statement generation
- Role-based access control

This expansion was necessary to ensure that **data flows from scheduling → execution → payroll**, rather than existing as isolated modules.

3. System Architecture and Design

3.1. Technology Stack

The final system was implemented using a modern full-stack web architecture, combining both frontend and backend capabilities within a unified framework.

- **Frontend:** Next.js (App Router), React, Mantine UI
- **Backend:** Next.js API routes (full-stack approach)
- **Typescript:** Strongly Typed Full-Stack Development
- **Database:** PostgreSQL with Prisma ORM
- **Authentication:** NextAuth (JWT-based)
- **Scheduling Interface:** FullCalendar (for interactive calendar-based scheduling and visualization)
- **Email Services:** Resend (for notifications and system-generated communication)
- **File Uploads:** UploadThing (for handling image uploads related to jobs and visit notes)
- **Deployment:** Vercel
- **AI Integration:** OpenAI-based structured assistance

This stack was selected to balance development efficiency, scalability, and maintainability. The use of Next.js as a unified full-stack framework enabled co-location of frontend and backend logic, reducing integration overhead and simplifying deployment.

Additional tools such as FullCalendar enhanced the usability of scheduling workflows through a visual interface, while services like Resend and UploadThing supported essential operational features such as notifications and documentation.

Overall, the architecture reflects a practical approach to building a production-ready system within real-world constraints, focusing on reliability, ease of deployment, and extensibility.

3.2. Database Design

Database Design

There was a significant database redesign, merging two separate schema versions into one unified model (mine and Vidarshan's). This was one of the most intensive phases of the project, as it required stabilizing the database while continuously adapting to evolving system requirements.

The process was challenging due to:

- Prisma migration conflicts
- schema mismatches between development branches
- continuous addition of fields and relationships to support UI and business logic

The database schema was ultimately redesigned to support the **entire operational workflow**, rather than isolated features. The finalized schema is structured to not only support current functionalities but also capture data for future analytics and decision-making.

The main entities are grouped as follows:

Entities

Core Business Operations Entities

- **Clients** – Represents customers served by EcoClean
- **Addresses** – Stores service locations linked to clients
- **Jobs** – Defines work to be performed (one-time or recurring)
- **Appointments** – Scheduled time slots for jobs (core scheduling unit)
- **Recurrence** – Handles repeating job patterns (daily, weekly, etc.)
- **Job Line Items** – Breakdown of services and pricing per job
- **Job Notes** – Operational notes such as access instructions, safety details, and client preferences
- **Visit Notes** – Post-appointment observations recorded by staff
- **Images (Appointment / Notes)** – Stores uploaded images for documentation and verification

These entities form the **core service delivery layer**, linking customers, locations, and work execution.

Staff Management Entities

- **Users** – Central entity for authentication and role-based access (Admin, Staff, Client)
- **Staff Profile** – Stores employment-specific data such as position and hourly rate
- **Staff Address** – Residential or contact address of staff
- **Emergency Contact** – Linked to staff for safety and HR purposes
- **Bank Details** – Stores payroll-related banking information
- **TD1** – Stores tax-related claim information (federal and Quebec)
- **Staff Availability** – Tracks availability by day and shift with effective dates
- **Leave** – Manages staff leave requests and approval workflow

This layer ensures separation between **authentication (User)** and **employment data (StaffProfile)**, improving flexibility and scalability.

Scheduling & Assignment

- **Assignments** – Links staff to appointments, defining who is responsible for each job
- Supports:
 - multiple staff per appointment
 - assignment status tracking (e.g., pending, on-site, completed)
 - planned vs actual work context
- **Appointment Work Sessions** – Tracks actual work performed (clock-in / clock-out)
 - Provides real-time work tracking
 - Serves as the foundation for accurate timesheets and payroll

This section bridges **planned scheduling** → **actual execution**, which is critical for operational accuracy.

Payroll & Time Tracking

- **Timesheet Periods** – Defines biweekly payroll cycles
- **Timesheets** – One per staff per period, with status flow (open → submitted → approved → locked)
- **Timesheet Day** – Tracks daily work entries and hours worked
- **Pay Statements** – Stores finalized payroll outputs including earnings, deductions, and net pay

Additional design considerations:

- Snapshot storage for payroll calculations ensures **historical integrity**
- Locking mechanism prevents modification after approval
- Denormalized totals improve performance for reporting

This structure ensures that payroll is **derived from actual recorded work**, rather than manual entry alone.

AI and Extended Features

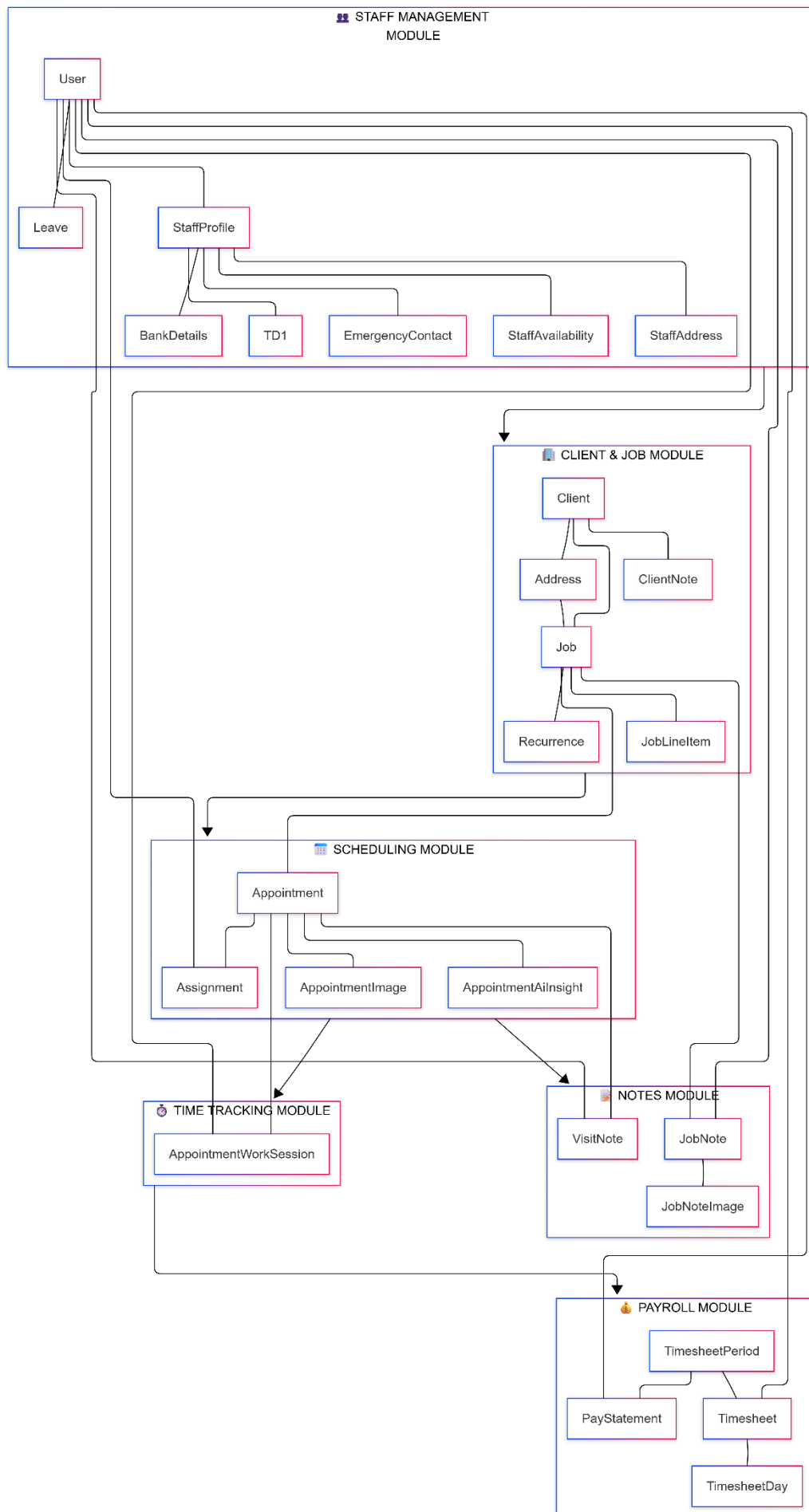
Appointment AI Insights - AI-generated recommendations and insights linked to appointments get stored, which will allow AI outputs to be stored, audited, versioned rather than being transient or untraceable.

Design Rationale

The database was designed with the following key principles:

- **Separation of concerns** (User vs StaffProfile vs Payroll)
- **Traceability** (from job → appointment → work session → payroll)
- **Data integrity** (use of constraints, relations, and locking mechanisms)
- **Scalability** (support for future analytics and SaaS expansion)

Overall, the schema reflects a transition from a feature-based design to a **workflow-driven architecture**, where all entities are connected to support the full lifecycle of operations.



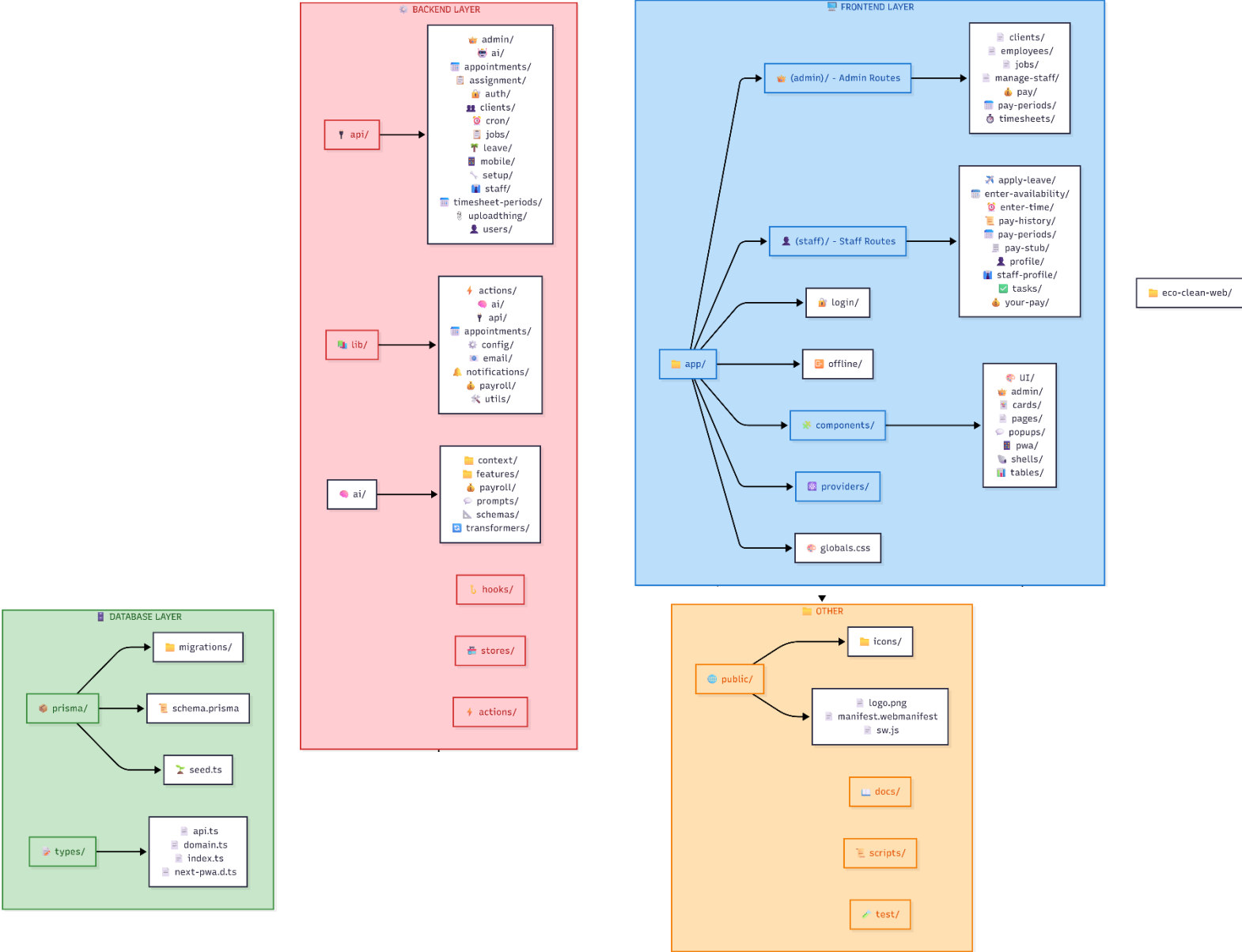
3.3. Eco-Clean Web Codebase Structure

3.3.1. High-Level View

Eco-Clean Web is a full-stack **Next.js App Router** application. That means the project keeps both the browser-facing interface and the server-side HTTP routes in the same repository. It is not split into separate frontend and backend repositories. Instead, the codebase is organized by responsibility:

- **Frontend:** pages, layouts, reusable UI components, modals, tables, and PWA navigation.
- **Backend:** API route handlers under `app/api`, authentication logic, server-side business logic, and database access.
- **Data Layer:** Prisma schema, migrations, seeds, and typed domain models.
- **Client Data Access:** browser API helpers and React Query hooks.
- **AI Layer:** model runtime, schemas, prompts, context builders, transformers, and AI API routes.
- **Utilities and Support:** notifications, date/time helpers, file upload support, payroll calculation helpers, and shared infrastructure.

3.3.2. Folder Structure



3.3.3. Frontend Structure

``app/`` as the UI Entry Layer

In Next.js App Router, the `app/` directory is the main entry point for the user interface. It contains pages, layouts, route groups, global styles, and route handlers. In this project, it serves two purposes at once: it defines what the user sees in the browser and it also contains the server endpoints under `app/api`.

The most important frontend segmentation inside `app/` is the separation into two route groups:

- ``app/(admin)/admin/`` contains the administrator-facing interface.
- ``app/(staff)/staff/`` contains the staff-facing interface.

This split is a clean design decision because the system has two very different user journeys. Administrators manage records, schedules, payroll, and approvals. Staff users primarily consume assigned work, submit operational data, and view their own records. Grouping these routes separately makes role-specific layouts and navigation easier to maintain.

Admin Frontend

The admin route group contains pages for clients, employees, jobs, staff management, pay, pay periods, and timesheets. This is the operational management dashboard of the application. It is the closest part of the system to a business administration portal.

Folder / File	Purpose
<code>app/(admin)/admin/page.tsx</code>	Main admin landing page or dashboard entry.
<code>app/(admin)/admin/clients/</code>	Client listing and client detail management screens.
<code>app/(admin)/admin/jobs/</code>	Job detail pages and scheduling-related interfaces.
<code>app/(admin)/admin/manage-staff/</code>	Staff administration, payroll pages, and leave approval workflows.
<code>app/(admin)/admin/pay-periods/</code>	Pay-period management screens.
<code>app/(admin)/admin/timesheets/</code>	Timesheet review and approval interface.
<code>DashboardShell.tsx</code> , <code>DashboardClient.tsx</code>	Compose the main admin dashboard UI.

Staff Frontend

The staff route group is closer to a field-work application. It includes task pages, leave application, availability entry, pay history, current pay, time entry, and profile management. This route group benefits from the PWA shell because the intended usage pattern is mobile and task-oriented.

Folder / File	Purpose
app/(staff)/staff/tasks/	Assigned tasks list and task detail execution pages.
app/(staff)/staff/enter-time/	Time-entry or time-submission interface.
app/(staff)/staff/enter-availability/	Availability submission UI.
app/(staff)/staff/apply-leave/	Leave request submission screen.
app/(staff)/staff/pay-history/, pay-periods/, pay-stub/, your-pay/	Staff self-service pay information pages.
app/(staff)/staff/profile/ and staff-profile/	Profile and employment details screens.

Reusable Frontend Components

The app/components/ directory contains reusable visual building blocks. This directory is important because it keeps page files from becoming too large and creates a separation between page composition and shared UI logic.

Subfolder	Role in the Frontend
UI/	Generic presentational components such as loaders.
admin/	Admin-only layout helpers such as page frames.
cards/	Compact reusable information blocks such as appointment cards and AI suggestion cards.
pages/	Higher-level page content components such as job details and staff profile pages.
popups/	Modal and popup workflows for editing clients, jobs, users, financial details, and appointment information.
pwa/	PWA-specific navigation such as the top bar and bottom bar.
shells/	Shell components that wrap sections such as client, staff, and user pages.
tables/	Tabular admin UIs such as staff and client tables.

From a frontend-architecture perspective, this is a good structure because it allows the application to scale horizontally. New pages can reuse cards, modals, shell patterns, and tables instead of rebuilding everything inside individual route files.

3.3.4. Backend Structure

``app/api/`` as the Server Boundary

The backend in this project lives primarily under `app/api/`. Each folder corresponds to an HTTP route, and each `route.ts` file defines the server-side logic for requests such as GET, POST, PATCH, or DELETE. This means the project uses Next.js route handlers as its backend layer instead of a separate Express or NestJS server.

The API directory is organized by business domain:

- ``admin/`` for privileged administrative actions such as payroll, leave approvals, pay statements, and timesheet review.
- ``ai/`` for AI-backed endpoints such as staff recommendation and task assistance.
- ``appointments/`` for appointment CRUD and appointment lifecycle actions like start, pause, complete, and visit-note submission.
- ``assignment/`` for staffing assignments and assignment-candidate logic.
- ``auth/`` for authentication routing through NextAuth.
- ``clients/``, ``jobs/``, ``users/``, and ``staff/`` for core business entities.
- ``cron/`` for scheduled maintenance or notification tasks such as reminders.
- ``uploadthing/`` for file-upload integration.

This design makes the backend understandable because each API area corresponds closely to a real business capability rather than an abstract technical layer.

Backend Logic Outside ``app/api/``

Not all backend logic is stored directly inside route handlers. The repository also uses the `lib/` folder to hold shared server-side logic. This is a good pattern because route handlers should mainly coordinate requests, authorization, validation, and response formatting, while reusable business logic should live in shared modules.

3.3.5. Data Access Layer

`lib/api/` for Browser-Side API Calls

The `lib/api/` directory acts as the client-side transport layer. These files contain helper functions for calling the backend routes. Instead of writing raw `fetch()` calls throughout the UI, the project centralizes request logic in one place.

This is important because it makes the frontend easier to maintain. If an endpoint changes, the request logic can often be updated in one helper module instead of in many separate page components.

`hooks/` for React Query Integration

The `hooks/` directory contains custom hooks such as `useJobs`, `useJob`, `useAppointmentDetails`, `useClient`, and `useStaff`. These hooks wrap browser-side requests in React Query. In practice, this means they handle loading state, caching, refetching, and invalidation in a standardized way.

Architecturally, this means the project follows a layered client pattern:

1. UI components call custom hooks.
2. The hooks call request helpers in `lib/api`.
3. The request helpers call server routes under `app/api`.
4. The server routes use shared logic and Prisma to talk to the database.

This is one of the strongest structural patterns in the repository because it keeps page components thin and data flow predictable.

3.3.6. Core Shared Library (`lib/`)

The `lib/` directory is the project's shared logic layer. It contains infrastructure code that is reused across many pages and routes. This folder is the key to understanding how the app avoids duplication.

Subfolder / File	Architectural Role
<code>lib/prisma.ts</code>	Creates or exports the Prisma client used for database access.
<code>lib/session.ts</code> , <code>lib/auth.ts</code> , <code>lib/auth-options.ts</code> , <code>lib/authorize.ts</code>	Authentication, session retrieval, and authorization support.
<code>lib/query-provider.tsx</code> , <code>lib/queryKeys.ts</code>	React Query setup and cache-key organization.
<code>lib/dateTime.ts</code>	Centralized timezone and date handling.

lib/appointments/	Appointment-specific shared logic such as reminders.
lib/payroll/	Payroll calculations and payroll-related reusable logic.
lib/email/	Email client setup and reusable email templates.
lib/notifications/	Notification helpers, including local notifications for the PWA workflow.
lib/uploadthing.ts	Shared upload support for appointment or note images.
lib/utils/	Low-level reusable utilities such as date formatting, distance calculation, and parsing helpers.

3.3.7. AI Architecture

The AI subsystem is intentionally separated inside lib/ai/ and app/api/ai/. This is one of the clearest architecture decisions in the codebase.

Folder / File	Purpose
lib/ai/client.ts, lib/ai/runtime.ts	Model client initialization and runtime behavior.
lib/ai/schemas/	Zod schemas that define the exact structure of AI outputs.
lib/ai/prompts/	Prompt templates and AI instructions.
lib/ai/context/	Builds deterministic application context before calling the model.
lib/ai/features/	Feature-level configuration such as task assistant setup.
lib/ai/transformers/	Transforms or normalizes AI-related payloads.
lib/ai/payroll/	Payroll-oriented AI support.
app/api/ai/	HTTP endpoints that expose AI features to the frontend.

This architecture is strong because it treats AI as a subsystem, not a scattered set of one-off calls. Prompts, schemas, runtime code, and HTTP routes are kept separate, which improves traceability and maintainability.

3.3.8. Database and Persistence Layer

``prisma/``

The `prisma/` directory is the database contract of the application. It contains:

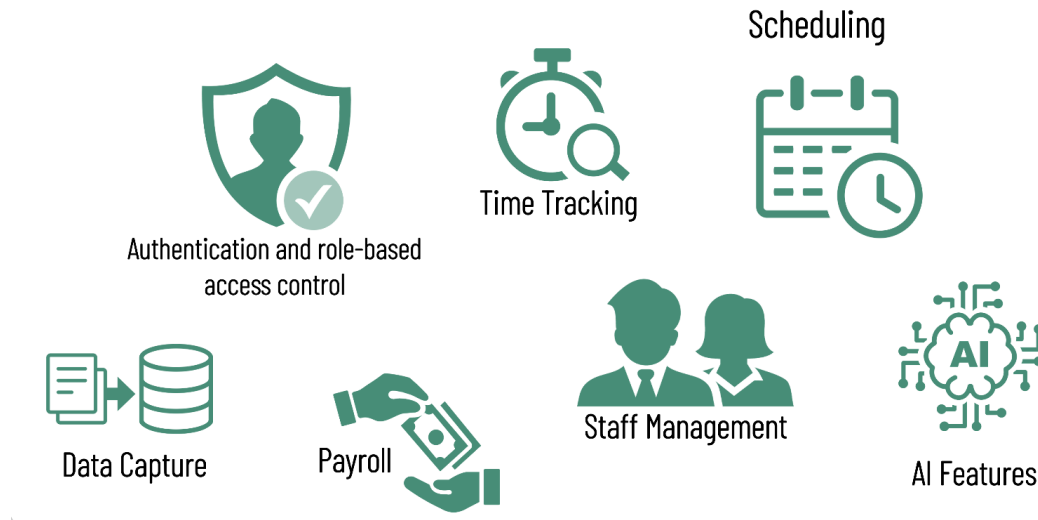
- ``schema.prisma`` for the data model.

- ``migrations/`` for incremental schema changes over time.
- ``seed.ts`` for initial or sample data generation.

This folder is where the domain becomes concrete. Entities such as users, clients, addresses, jobs, appointments, staff profiles, leaves, assignments, work sessions, timesheets, and pay statements are defined here. Every major backend route eventually relies on this layer.

The migration history also tells a story about project growth: client tables were added early, scheduling was refined, work sessions were introduced, assignment tables were added, AI insight persistence was added, and staff financial tables appeared late in the project. So the Prisma folder documents not only structure but also the evolution of the application.

4. Key Features and Functionalities



Scheduling and Assignment

The system supports:

- One-time and recurring jobs
- Appointment creation with time slots
- Multi-staff assignment

This forms the **foundation of the operational workflow**.

Staff Management

A complete staff management module was implemented including:

- Staff profiles with structured data
- Emergency contacts and addresses
- Role-based access (Admin vs Staff)
- Self-service profile updates
- This ensured both central control and individual accessibility .
-
- Time Tracking and Work Sessions
- Instead of relying only on manual timesheets, the system introduced:
- Clock-in / clock-out tracking
- Work session recording
- Comparison between scheduled vs actual work

This significantly improves payroll accuracy by linking **actual work activity to earnings**.

Payroll and Pay Statements

The payroll system was one of the most advanced components:

- Biweekly pay periods
- Earnings calculated from hours $\times$ rate
- CRA-based deduction estimation
- Manual override capability for admins

The system integrates:

- Federal tax
- Quebec tax
- QPP, EI, QPIP

These were implemented manually due to lack of free APIs .

Staff-Facing PWA

The PWA allows staff to:

- View assigned jobs
- Track work sessions
- Submit timesheets
- Apply for leave
- View pay statements

This ensures **real-time interaction with the system in the field**, without requiring a native app.

AI Integration

AI was introduced as a **decision-support tool**, not a decision-maker.

Key principles followed:

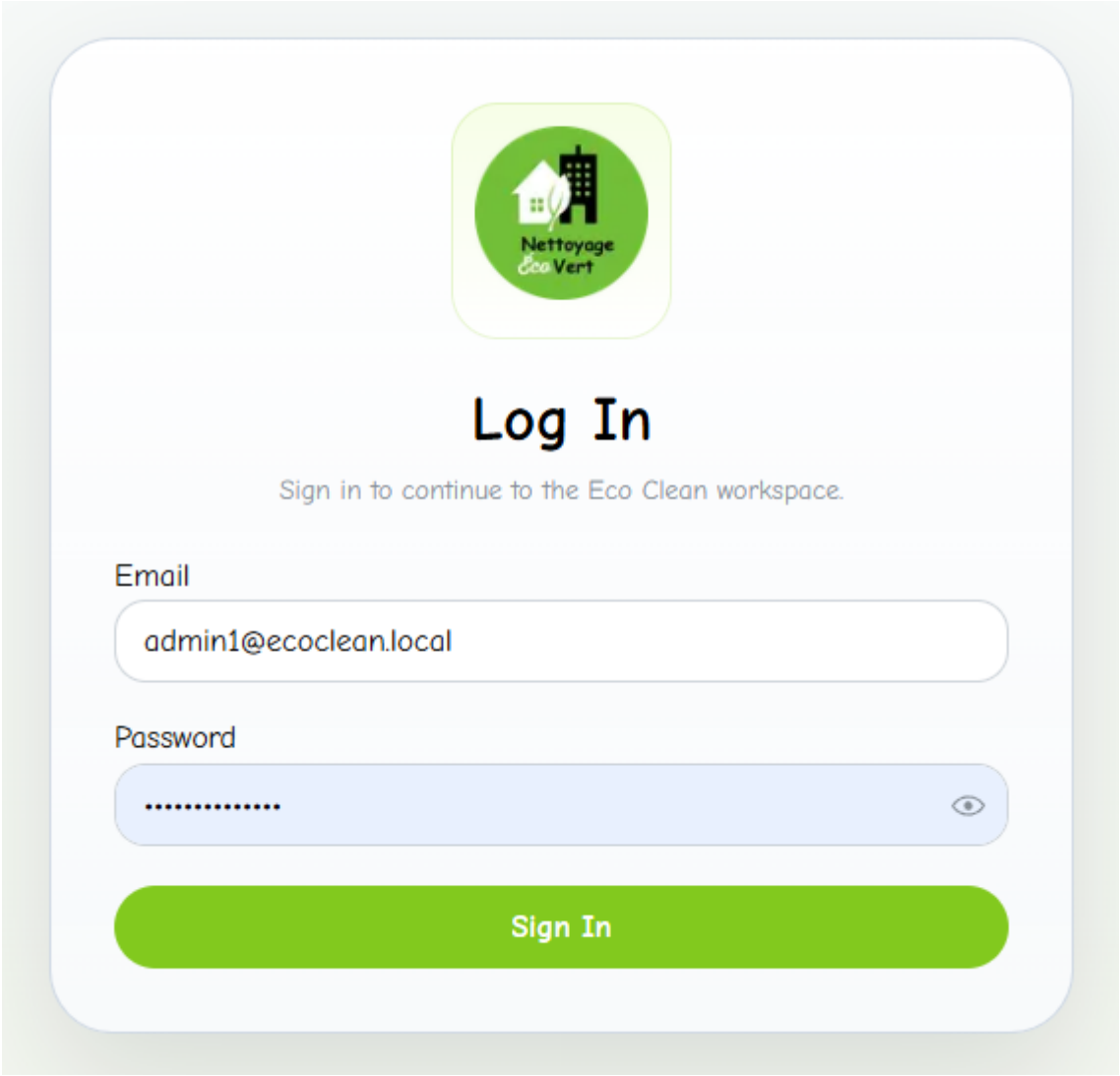
- AI uses structured system data
- Outputs are validated before use
- AI assists in suggestions (e.g., planning, staff selection)

Unlike typical chatbot integrations, this approach ensures:

- Data consistency
- Auditability
- Controlled automation

4.1. Platform Interfaces

Login Interface: (Admin or Staff can login)



The login interface is a white rounded rectangle centered on a light gray background. At the top center is a circular logo with a green background, containing a white house icon and a green leaf, with the text "Nettoyage Eco Vert" below it. Below the logo is the title "Log In" in a large, bold, black font. Underneath the title is the subtitle "Sign in to continue to the Eco Clean workspace." in a smaller, gray font. The form contains two input fields: an "Email" field with the text "admin1@ecoclean.local" and a "Password" field with masked characters ".....". To the right of the password field is an eye icon for toggling visibility. At the bottom of the form is a large, rounded green button with the text "Sign In" in white.



Log In

Sign in to continue to the Eco Clean workspace.

Email

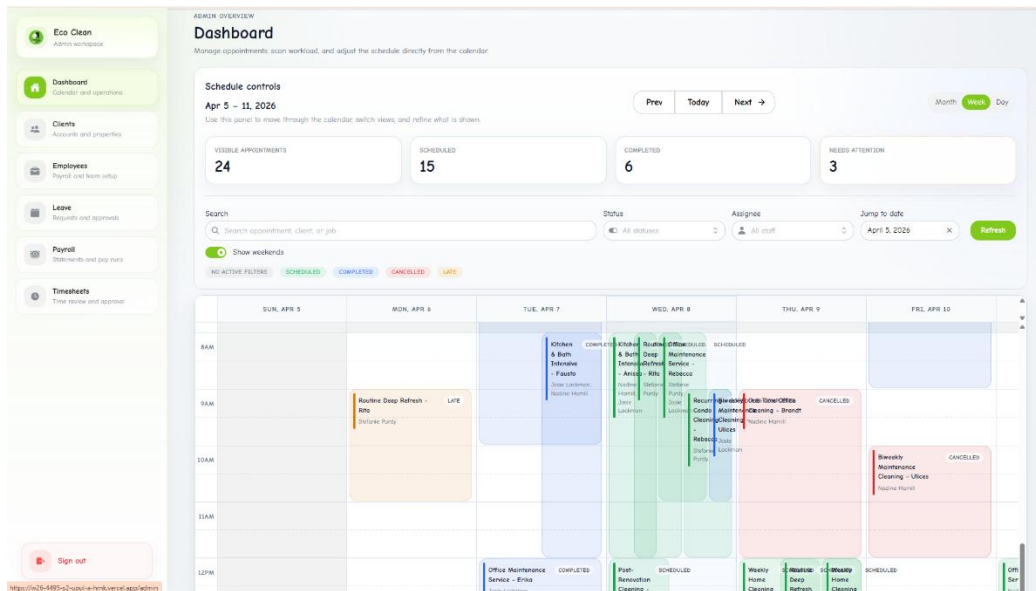
admin1@ecoclean.local

Password

..... 

Sign In

4.1.1. Admin Interfaces

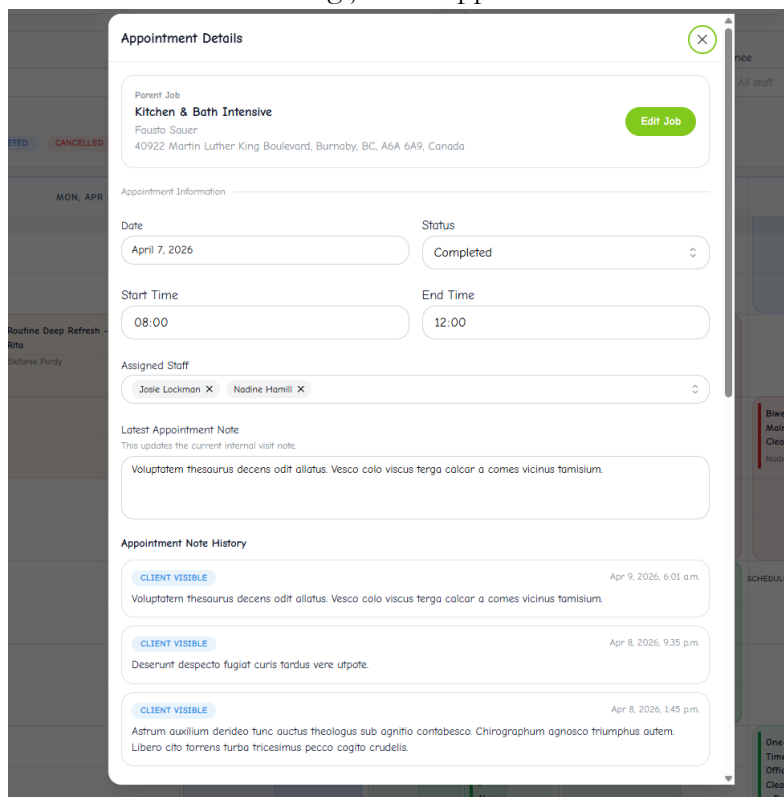


Dashboard

The dashboard was created with a calendar view to showcase the existing appointments. The buttons on the leftside on the dashboard shell will provide various routes to admin user.

The Job Modal

The admin users can view and edit the existing jobs or appointments.



Client Workspace

This interface helps administrators to manage client accounts and property records.

Eco Clean
Admin workspace

Dashboard
Calendar and operations

Clients
Accounts and properties

Employees
Payroll and team setup

Leave
Requests and approvals

Payroll
Statements and pay runs

Timesheets
Time review and approval

CLIENT WORKSPACE

Clients

Manage client accounts and property records

Client directory

Search, sort, and open client records without the page title competing with the working area below.

CLIENT RECORDS

10 SHOWN

Add client

Search clients

Newest

Client	Company	Email	Phone	Preferred	Lead source	Created
Ulises Murphy	Crona, Cole and Gattlieb	ulises_murphy95@yahoo.com	(604) 166-8236	EMAIL	Flyer	4/8/2026
Monte Feest	—	montefeest@gmail.com	(778) 433-3168	EMAIL	Referral	4/8/2026
Alicia Waelchi	—	alicia_waelchi@gmail.com	(778) 217-2745	PHONE	Google	4/8/2026
Jerald Stark	—	gerald_stark@gmail.com	(604) 472-7998	EMAIL	Facebook	4/8/2026
Hannah Pagac	—	hannah.pagac@hotmail.com	(236) 468-1193	TEXT	—	4/8/2026
Raul Kreiger	—	raul.kreiger47@yahoo.com	(778) 389-9950	PHONE	Instagram	4/8/2026
Alanna Weber	Hand-Kemmer LLC	alanna_weber@hotmail.com	(672) 370-1332	PHONE	—	4/8/2026
Fausto Sauer	—	fausto_sauer@hotmail.com	(672) 277-1496	PHONE	Google	4/8/2026
Pink Breitenberg	—	pink.breitenberg@yahoo.com	(672) 542-3495	TEXT	Website	4/8/2026
Anissa Trantow	Barton - Howell	anissatrantow@yahoo.com	(672) 872-9883	EMAIL	Referral	4/8/2026

Team Workspace

Eco Clean
Admin workspace

Dashboard
Calendar and operations

Clients
Accounts and properties

Employees
Payroll and team setup

Leave
Requests and approvals

Payroll
Statements and pay runs

Timesheets
Time review and approval

TEAM WORKSPACE

Employees

Review employee records and open staff profiles

Employee directory

Search, sort, and open employee records from one consistent working surface.

EMPLOYEE RECORDS

8 SHOWN

Add user

Search users

Newest

Name	Role	Email	Joined
Admin	ADMIN	admin3@ecoclean.local	4/8/2026
John Doe	STAFF	johndoe@gmail.com	4/8/2026
Test User	STAFF	test@gmail.com	4/8/2026
Stefanie Purdy	STAFF	staff2@ecoclean.local	4/8/2026
Brionna O'Reilly	STAFF	staff3@ecoclean.local	4/8/2026
Nadine Hamill	STAFF	staff4@ecoclean.local	4/8/2026
Josie Lockman	STAFF	staff1@ecoclean.local	4/8/2026
Admin 1	ADMIN	admin1@ecoclean.local	4/8/2026

This interface review employee records and open staff profiles

NEW USER

USER ACCESS

Create a team account

Control identity, role access, and staff-profile management from one admin entry point.

Account details

Name

Aa Staff name

Email

Aa Staff email

Role

Staff

Cancel

Create

EXISTING USER

Account details

Name

Aa John Doe

Email

Aa johndoe@gmail.com

Role

Staff

Password reset

Reset password

Leave blank to keep unchanged

Confirm password

Re-enter password

Staff workspace

Open profile, payroll, and contact management in one larger admin panel.

Open Staff Workspace

Delete

Cancel

Save

John Doe

STAFF ID: STF-ECO-5323

Review staffing, contact, and payroll-related details in one place.

Overview

Financial

Role and pay

This section controls role-adjacent profile values used in staffing and payroll.

Position

Hourly Rate

\$0

Email

Phone

johndoe@gmail.com

No contact number

Position

Hourly Rate

—

\$0.00

Address

Current home address on file.

No address on file.

Emergency Contact

Primary contact in case something goes wrong on shift.

No emergency contact on file.

Close

Save

Leave Management Portal

Eco Clean

Admin workspace

Dashboard

Calendar and oper...

Clients

Accounts and prop...

Employees

Payroll and team s...

Leave

Requests and appr...

Payroll

Statements and pa...

Timesheets

Time review and a...

LEAVE MANAGEMENT

Leave Requests

Review pending requests quickly, keep processed items visible, and make approval decisions without scanning a crowded page.

Pending

2

Approved

5

Rejected

0

Approval list

Each row shows the request window, leave type, current status, and approval actions.

Employee

Leave type

Dates

Reason

Status

Actions

Nadine Hamill

UNPAID SICK

May 4, 2026 to May 7, 2026

Illum consuasor absum candidus ipsom trans tabernus.

APPROVED

Approve Reject

Nadine Hamill

VACATION

Apr 23, 2026 to Apr 26, 2026

Antiquus apparatus celebrer creta villis beatus.

PENDING

Approve Reject

Josie Lockman

PAID SICK

Mar 26, 2026 to Mar 29, 2026

No reason provided

APPROVED

Approve Reject

Nadine Hamill

UNPAID SICK

Mar 10, 2026 to Mar 13, 2026

Confido cum abstergo cunctatio statim censura abundans ossidus ratione vesco.

PENDING

Approve Reject

Brionna O'Reilly

PAID SICK

Feb 12, 2026 to Feb 13, 2026

Exercitationem vestibulum necessitatibus cupiditate upote taceo admitta.

APPROVED

Approve Reject

Josie Lockman

PERSONAL

Feb 14, 2026 to Feb 15, 2026

No reason provided

APPROVED

Approve Reject

Josie Lockman

PAID SICK

Feb 8, 2026 to Feb 8, 2026

Solutio cena tonsor talis bonus curo timor toedium.

APPROVED

Approve Reject

23

Time Tracking and Timesheets Approval

Eco Clean
Admin workspace

Dashboard
Calendar and operations

Clients
Accounts and properties

Employees
Payroll and team setup

Leave
Requests and approvals

Payroll
Statements and pay runs

Timesheets
Time review and approval

TIME TRACKING

Timesheets

Compare planned work against submitted time, then approve ready timesheets from a more readable admin review flow.

Employees
4

Awaiting approval
3

Period variance
88h 0m

Timesheet review window

Review one period at a time, then approve submitted staff timesheets from the employee cards below.

OPEN

DEC 27, 2026 → DEC 31, 2026

Dec 27, 2026 → Dec 31, 2026 (OPEN)

Locked at —

Employees
4

Planned
0h 0m

Actual
88h 0m

Variance
+88h 0m

Brionna O'Reilly

staff3@ecoclean.local

Staff ID: EC-0003 | Position: Cleaner

OPEN

Collapse

Planned
0h 0m

Actual
23h 0m

Variance
+23h 0m

2026-12-28

No timesheet note

+8h 0m

Planned
0h 0m

Actual
8h 0m

Hourly rate
\$28.89

Assignments
0

No planned assignments

Once a staff member submit their time sheet, it will be directly reflected in here for the management to approve. This would be the middle step of preparing the payroll functions.

Payroll Interface

PAYROLL ADMIN

Pay Periods

Review approved timesheet totals, fine-tune deductions, and generate pay statements from the newer admin page system.

Generate statements

Export Excel

Selected period
1 active

Staff rows
4

Net payroll
\$1,102.96

Payroll run setup

Choose one approved period, review the imported staff rows, then generate statements from this same screen.

MAR 22, 2026 - APR 4, 2026

Payroll rows

Each employee row is grouped into smaller edit sections so you can work without a wide spreadsheet view.

Josie Lockman
EC-0001

Reset deductions

Earnings

Regular hours
80

Regular rate
29.42

OT hours
3

OT rate
44.13

Transport
50

Payroll deductions

Federal
505.06

Quebec
583.19

EI
38.53

QPP
265.552

QPP2
0

QPIP
18.7

Other and totals

Health
22

Other
0

Regular amount
\$2,353.60

OT amount
\$132.39

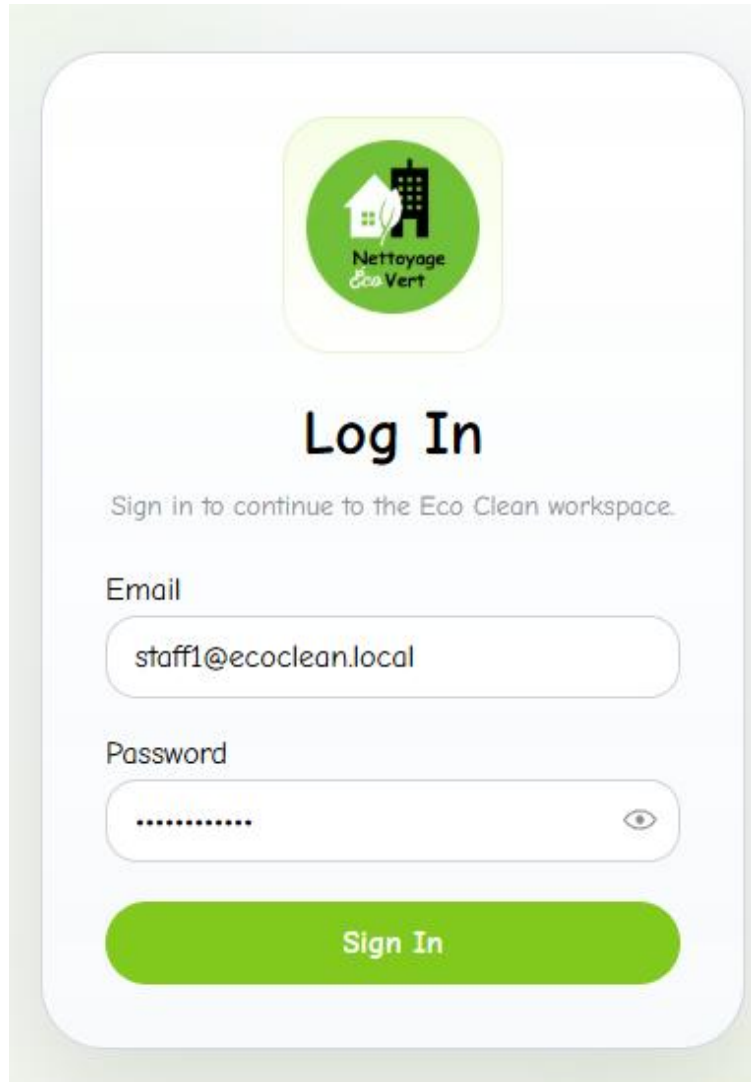
Deductions
\$1,433.03

Gross
\$2,535.99

4

The approved timesheets will be reflected in here for the final payroll calculations. Most of the payroll calculations are automated. Once the administrator can double check this, they would be able to generate the paystatements, which can be accessed via the PWA interfaces.

4.1.2. PWA Interfaces



The screenshot shows a login interface for a PWA application. At the top, there is a circular logo with a green background, featuring a white house icon and a black building icon, with the text "Nettoyage Eco Vert" below it. Below the logo, the text "Log In" is displayed in a large, bold, black font. Underneath "Log In", there is a smaller text prompt: "Sign in to continue to the Eco Clean workspace." Below this prompt, there are two input fields. The first is labeled "Email" and contains the text "staff1@ecoclean.local". The second is labeled "Password" and contains a series of dots, with a small eye icon to its right. At the bottom of the form, there is a large, rounded green button with the text "Sign In" in white.

Once a staff member logged in the will see the following screens which will enable them viewing the tasks, time logging, staff detail management, leave management, timesheet submission and view their pay.

My Tasks



Eco Clean
Staff workspace



My Tasks
Assignments and updates



Profile
Personal and job details



Your Time
Timesheets and entries



Your Pay
Statements and history



Time-off
Leave requests



Availability
Schedule preferences



Logout

5 TOTAL

Completed

>

Su

5

12

19

26

3

SCHEDULED

minster, BC

SCHEDULED



Tasks



Your Tasks

Wednesday, Apr 8

4 ACTIVE5 TOTAL

Upcoming

Cancelled

Completed

April 2026

<

April 2026

>

Mo	Tu	We	Th	Fr	Sa	Su
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3

Office Maintenance Service

SCHEDULED

Wednesday, Apr 8

 8:00 AM - 11:00 AM

 Rebecca Lind

 98751 Mercedes Bridge, New Westminster, BC

Kitchen & Bath Intensive

SCHEDULED

Wednesday, Apr 8

26

The staff can:

- View assigned jobs and task details
- Start, pause, complete tasks
- Shows time tracking

Tasks

Your Tasks

Wednesday, Apr 8

4 ACTIVE 5 TOTAL

Upcoming Cancelled Completed

April 2026

< April 2026 >

Mo	Tu	We	Th	Fr	Sa	Su
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3

Office Maintenance Service

SCHEDULED

Wednesday, Apr 8

8:00 AM - 11:00 AM

Rebecca Lind

98751 Mercedes Bridge, New Westminster, BC

Task Details

JOB FOR TODAY

Office Maintenance Service

Wednesday, Apr 8

RECURRING

Not started

REMAINING

02:59:47

00:00:13 used out of 03:00:00

0% of your planned time used

Start 8:00 AM End 11:00 AM

Start Job Pause Job

Complete Job

AI Overview Expand

Team Activity

0 of 2 working right now

Stefanie Purdy NOT STARTED

- Add notes/images

- AI Overview on the task

 **AI Overview**

Collapse

Quick Brief

Two-person office maintenance required for a mid-morning visit; no staff are on leave and current assignments do not conflict with an 8:00–11:00 service. Focus on high-traffic and washroom sanitization, efficient zone coverage, and on-time completion so afternoon commitments are met.

Priority Order

- Confirm building access & parking before departure
- Sanitize and restock restrooms first
- High-traffic surfaces and shared areas (kitchen, reception) next
- Vacuum/dust workstations and walkways
- Final walkthrough, client sign-off and photos

Suggested Time Plan

Arrival, access check & setup

20 min

Restrooms deep clean & restock supplies

45 min

Kitchen/communal surfaces, wipe & sanitize

30 min

 **Instructions**

There are no extra notes for this job.

 **Previous notes**

Visit note

Apr 8, 2026

Hi



 **Add a note**

What should the team know?

Add anything helpful from this visit.



Drag visit images here or click to upload (4MB Max, up to 10 images)

Save Visit Note

Profile Management

Profile

PERSONAL WORKSPACE

Profile

Keep your contact details up to date so the team can reach you when needed.

TEAM LEAD

STAFF ID EC-0001

Full name

Josie Lockman

Email

staff1@ecoclean.local

Phone

(604) 915-3597

Emergency contact

Davion Williamson

Edit Details

Personal details

Your basic contact details in one place.

Full name

Josie Lockman

Email

staff1@ecoclean.local

Profile

Postal code

A9A 5A4

Emergency contact

Davion Williamson • (604) 743-1801

Review Contact

Quick links

Go straight to the pages you use most often.

Enter Time

Review your current pay period and submit timesheets.

Pay History

Look back at older pay slips.

Availability

Choose the days and times you can work.

Apply Leave

Ask for time off.

Entering Time

Time

Enter Time

Check your hours for each day and send them in when they look right.

Current Week

Week 1

Week Total

104:29

Sessions

8

Pay Period

05/04/2026 to 18/04/2026

Week 1

Week 2

Sun

Apr 5

Logged hours

24:00

Time

Week summary

Sun

Apr 5

24:00

Mon

Apr 6

24:00

Tue

Apr 7

34:30

Wed

Today

21:59

Thu

Apr 9

0:00

Fri

Apr 10

0:00

Sat

Apr 11

0:00

Next Week

30

Your Pay

Eco Clean

Pay Statement

Employee

Employee ID

Pay Date

Ayesha2

STF-ECO-AKUK

4/6/2026

Earnings Distribution

Gross Earnings

\$736.00

Gross Earnings

\$736.00

Regular

\$736.00

Overtime

\$0.00

Allowance

\$0.00

YTD

\$2036.00

Total Deductions

\$152.82

Federal Tax

\$0.00

Quebec Tax

\$102.20

EI

\$9.57

QPP

\$37.89

QPIP

\$3.16

Other

\$0.00

YTD

\$489.14

Net Earnings

\$583.18

Eco Clean

Manage Pay Periods

Pay Date

4/6/2026

Period Start

4/5/2026

Period End

4/18/2026

Net Earnings

\$583.18

Details

Pay Date

4/2/2026

Period Start

3/22/2026

Period End

4/4/2026

Net Earnings

\$963.68

Details

A Staff Member can also download paystubs as pdf and compare their earnings across periods using AI insights.

AI Pay Insights

Hide

3/22/2026 - 4/4/2

4/5/2026 - 4/18/2

Compare

Summary

Net pay decreased from Period A to Period B. Gross pay fell by 564, deductions fell by 183.5, and net pay fell by 380.5.

Key Drivers

- Lower gross pay (grossDiff = -564) — primary driver of the lower net pay.
- Reduction in deductions (deductionDiff = -183.5) — partially offset the gross drop but net still decreased.
- Unknown hours/overtime (hoursDiff and overtimeDiff are null) — an unknown change in hours or overtime could explain the gross change and should be checked.

Increases

- + Deductions decreased (deductionDiff = -183.5), which partially helped take-home pay.

Decreases

- Gross pay decreased (grossDiff = -564).
- Net pay decreased (netDiff = -380.5).
- Hours and overtime data are missing (null) — confirm whether hours worked or overtime changed.
- Confirm there was no payroll error or missed payment because of the large gross decrease.

Download PDF

Nettoyage Eco-Vert

STATEMENT OF EARNINGS

Pay Period: 4/5/2026 - 4/18/2026

Employee: Ayesha2

Employee ID: STF-ECO-AKUK

Pay Date: 4/6/2026

Department: Cleaning Services

Earnings

	Amount	YTD
Regular	736.00	1536.00
Overtime	0.00	300.00
Transport	0.00	200.00
Gross Earnings	736.00	2036.00

Deductions

	Amount	YTD
Federal Tax	0.00	60.04
Quebec Tax	102.20	282.57
EI	9.57	26.47
QPP	37.89	111.31
QPP2	0.00	0.00
QPIP	3.16	8.75
Total Deductions	152.82	489.14

Net Earnings

583.18	1546.86
--------	---------

Availability and Time Off Management

Time-off

Your Schedule

View time off, check balances, and submit leave requests.

Balances

Request

Calendar

Tap a date to request leave

<

April 2026

>

Mo

Tu

We

Th

Fr

Sa

Su

29

30

31

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

1

2

Time Off Balances

Request time off

Date

April 8, 2026

Vacation remaining

80.0 hrs

Sick used

40.0 hrs

Recent requests

PAID SICK

26/03/2026

12:00 AM to 11:59 PM

PERSONAL

14/02/2026

12:00 AM to 11:59 PM

PAID SICK

08/02/2026

12:00 AM to 11:59 PM

Solutio ceno tonsor talis bonus curo timor taedium.

Availability

SCHEDULE PREFERENCES

Availability

Tell us which days and times you are usually available.

TWO SHIFTS PER DAY

Effective date

Your changes will start next week.

Shift pattern

Choose morning, afternoon, or both for each day.

Today's Date

April 8, 2026

Employee Name

Josie Lockman

Effective Date

When does this start?

Comments

Optional notes

Weekly shift availability

Pick the times you are usually free to work.

Day	Available?	Morning (S1)	Afternoon (S2)
Monday	☐	☒	☒
Tuesday	☐	☒	☒
Wednesday	☐	☒	☒
Thursday	☐	☒	☒
Friday	☐	☒	☒
Saturday	☐	☒	☒
Sunday	☐	☒	☒

Save Availability

32

5. Evaluation Techniques

The evaluation of the Eco-Clean Web system was conducted using a combination of **functional testing, workflow validation, and user feedback**, rather than purely theoretical benchmarks. Since the system is designed as an operational platform for a real service business, the evaluation focused on practical usability, correctness of workflows, and alignment with business needs.

5.1. Functional and Workflow Testing

The system was tested using seeded data to simulate real-world scenarios across all modules, including scheduling, staff management, time tracking, and payroll.

End-to-end workflows were validated to ensure that:

- jobs can be created and scheduled correctly
- staff can be assigned and tracked
- work sessions translate into timesheet data
- payroll calculations are generated from recorded work

This testing ensured that the system operates as a **connected workflow**, rather than isolated features.

5.2. Payroll Validation

Since payroll calculations were implemented manually based on CRA guidelines, additional validation was required to ensure accuracy.

The system outputs were:

- compared against expected calculation patterns
- reviewed for consistency across different input scenarios

Based on this evaluation:

- manual override options were introduced
- payroll fields were designed to allow admin adjustments

This ensured that the system remains **practically usable**, even if exact compliance requires external verification.

5.3. User Feedback and Client Evaluation

The system was demonstrated to the client, where key workflows such as scheduling, staff management, and payroll generation were presented.

Feedback was gathered based on:

- usability of the interface
- clarity of workflows

- alignment with actual business operations

Based on this feedback:

- the admin dashboard was restructured for better navigation
- staff-facing interfaces were improved for mobile usability
- certain workflows were simplified to reduce operational complexity

5.4. Usability and Interface Evaluation

The system was iteratively refined based on internal testing and observation of how users interact with the interface.

Key areas evaluated included:

- ease of navigation across modules
- clarity of scheduling and calendar views
- accessibility of staff-facing features on mobile devices

This led to:

- improved layout organization
- responsive design enhancements
- better alignment between UI and backend functionality

5.5. Deployment and Environment Validation

The system was also evaluated in a production-like environment through deployment on Vercel.

This exposed:

- environment configuration issues
- stricter type validation requirements
- differences between development and production behavior

Resolving these ensured that the system is **deployment-ready and stable**, not just functionally correct in a local environment.

6. Project Planning & Timeline

Phase / Task	Leading Person(s)*		15-Jan-26	22-Jan-26	29-Jan-26	05-Feb-26	12-Feb-26	19-Feb-26	26-Feb-26	05-Mar-26	12-Mar-26	19-Mar-26	26-Mar-26	02-Apr-26	09-Apr-26
1. DISCOVERY & ANALYSIS															
Understanding the Riipen project and its scope	Upul & Vidarshan	Planned													
		Actual													

7. Challenges Encountered

When dealing with the Implementation, we had run into the following issues.

7.1. Database Migration and Schema Conflicts

During development, two separate database schema versions were created independently and later needed to be merged into a single unified model. This process introduced multiple challenges, including migration conflicts, schema mismatches, and data inconsistencies across different environments.

Solution:

- Redesigned relationships and entities to ensure consistency across all modules
- Stabilized Prisma migrations by aligning schema definitions and resolving conflicts incrementally

7.2. Authentication Issues

In the early stages of development, authentication was temporarily bypassed to accelerate feature implementation. However, this resulted in inconsistencies between session data and actual database records, leading to incorrect role assignments and authorization conflicts.

Solution:

- Rebuilt the NextAuth configuration to properly handle authentication and session management
- Implemented strict role-based access control to ensure data security and correct system behavior

7.3. Deployment Issues (Vercel)

When deploying the application to Vercel, stricter build and runtime validations exposed several issues that were not apparent during local development. These included TypeScript errors and misconfigured environment variables.

Solution:

- Refined TypeScript usage to ensure type safety across all components and API routes
- Standardized environment variable management to maintain consistency between development and production environments

7.4. Payroll Complexity

The implementation of payroll functionality presented challenges due to the absence of freely available APIs for Canadian payroll calculations, particularly for Quebec-specific deductions.

Solution:

- Developed custom payroll calculation logic based on CRA guidelines and publicly available rules
- Incorporated manual override options to allow administrators to validate and adjust calculations based on real-world requirements

7.5. Time Zone Handling in Scheduling

During development, appointments were initially recorded using the local development time zone (PST). However, since the system is intended for use in Quebec (EST), this created inconsistencies in scheduling and time representation.

Solution:

- Standardized all date-time values to **UTC** at the database level
- Implemented dynamic conversion to the user's local time zone on the frontend

8. Conclusion

Eco-Clean Web demonstrates that a unified digital platform can significantly improve operational efficiency in service-based businesses.

The system successfully integrates:

- scheduling
- staff management
- time tracking
- payroll
- AI-assisted decision support

Beyond the implementation, this project provides a practical framework for how small businesses can adopt digital transformation in a cost-effective and scalable manner.

The solution also lays the foundation for future enhancements such as:

- analytics dashboards
- AI-driven optimization
- multi-company SaaS deployment

9. Future Improvements

- Advanced payroll compliance (full CRA integration)
- AI-driven scheduling optimization
- Multi-tenant SaaS architecture
- Enhanced reporting dashboards
- Mobile UX enhancements

10. External Partners & Affiliates

This project was sourced through the Riipen platform.

Project Link: <https://douglascollege.riipen.com/projects/rVnMYjLj>

Company: Nettoyage Eco Vert

Company Contact: Stephanie Riddell, Founder

Riipen Platform Contact: Morgan Brown

11. Work Logs

Member: Upul Atapattu

Date	Hours	Description
15th January to 23rd March	113.25 hours	Previous Work
24th March 8:30 pm – 10:30 pm	2 hours	Worked on improving payroll calculation logic by refining deduction formulas and aligning them with Quebec requirements. Fixed inconsistencies in tax calculations and validated outputs.
25th March 9:00 pm – 11:30 pm	2.5 hours	Focused on debugging Prisma schema and seed data mismatches. Resolved migration conflicts and ensured database alignment with application models.
26th March 8:00 pm – 10:00 pm	2 hours	Worked on Timesheet and Pay Period integration. Developed logic to fetch approved timesheets and improved backend API for payroll generation.
27th March 9:30 pm – 12:00 am	2.5 hours	Enhanced staff assignment and availability handling. Improved data fetching logic and fixed UI data mapping issues.
28th March 10:00 pm – 12:30 am	2.5 hours	Improved Admin Dashboard timesheet UI and fixed runtime errors related to undefined values and numeric handling.
29th March 7:30 pm – 10:30 pm	3 hours	Refined admin panel UI/UX, improving layout, navigation, and responsiveness across Leave, Payroll, and Timesheets.
30th March 8:30 pm – 11:00 pm	2.5 hours	Enhanced Staff Profile UI with better layout, editable sections, and improved modal interactions.
31st March 9:00 pm – 11:30 pm	2.5 hours	Resolved Next.js routing issues and ensured correct data loading for dynamic routes such as pay-stubs.
1st April 8:00 pm – 10:30 pm	2.5 hours	Improved Pay Stub generation and UI. Added support for latest paystub retrieval and dynamic routing.
2nd April 7:30 pm – 11:00 pm	3.5 hours	Fixed TypeScript and runtime errors, improving stability and handling undefined values across components.
3rd April 9:00 pm – 11:30 pm	2.5 hours	Worked on poster design, refining content, layout, and academic formatting.

4th April 8:30 pm – 11:00 pm	2.5 hours	Enhanced payroll UI with export functionality and improved action buttons and layout spacing.
5th April 7:30 pm – 10:30 pm	3 hours	Fixed multi-select and staff data loading issues by debugging API integration.
6th April 8:00 pm – 11:30 pm	3.5 hours	Refined payroll deduction logic aligned with Quebec rules and improved manual override handling.
7th April 9:00 pm – 11:30 pm	2.5 hours	Worked on GitHub workflows, pull requests, and report improvements.
8th April 8:30 pm – 11:30 pm	3 hours	Finalized report sections and improved presentation materials and formatting.
Total	155.75 hours	

Member: Vidarshan Rathnayake

Date	Hours	Description
15th January to 23rd March	151 hours	Previous Work
25th March 7:30pm–10:30pm	3 hours	Redesigned admin and staff UI across dashboard, tasks, payroll, and profile pages
26th March 5:30pm–7:00pm	1.5 hours	Integrated AI features into job modal and appointment workflows
26th March 7:00pm–9:00pm	2 hours	Refactored application to centralize state management and data querying
26th March 9:00pm–10:00pm	1 hour	Implemented AI support within job modal and appointment flow
27th March 6:00pm–7:30pm	1.5 hours	Fixed task appointment detail issues and improved data handling
2nd April 9:00am–11:00am	2 hours	Improved login and authentication flow across admin and staff pages
2nd April 5:00pm–8:00pm	3 hours	Refined PWA experience for tasks, availability, and time entry screens
3rd April 6:00pm–8:30pm	2.5 hours	Designed and prepared research poster for project presentation

5th April 12:00am–2:00am	2 hours	Finalized staff task screen and improved layout and AI assistant integration
5th April 2:00am–3:00am	1 hour	Cleaned React and TypeScript warnings in task and AI components
5th April 3:00am–5:00am	2 hours	Implemented AI task assistant feature within task workflows
5th April 10:00am–11:30am	1.5 hours	Documented AI feature workflow for reporting and presentation
6th April 6:00pm–7:30pm	1.5 hours	Applied cosmetic UI improvements across dashboard and modals
7th April 11:00am–3:30pm	4.5 hours	Implemented Job Edit Modal with API integration, resolved merge conflicts, and fixed timezone issues
7th April 7:00pm–10:30pm	3.5 hours	Stabilized UI across admin and staff pages and improved overall application consistency
7th April 10:00am–1:00pm	3 hours	Improved PWA user experience and applied UI consistency fixes
8th April 2:00pm–4:00pm	2 hours	Resolved lint issues, updated documentation, and fixed minor data handling issues
Total	188.5 hours	

12. AI Usage

Member: Upul Atapattu

AI Model	AI Prompt	Context / Purpose / Value Addition
ChatGPT 5.3 Plus	Why is my timesheet showing data but payroll page is empty even though both use same period?	Debugging payroll data linkage
ChatGPT 5.3 Plus	In my payroll page.tsx, how can I ensure it always fetches APPROVED timesheets only?	Fixing payroll filtering logic
ChatGPT 5.3 Plus	My paystub is loading but name and staffId are undefined. Where should I fix this?	Debugging data binding issues
ChatGPT 5.3 Plus	How to map nested Prisma data (user → staffProfile) correctly in React?	Fixing relational data handling
ChatGPT 5.3 Plus	Why does my Prisma seed fail even after migration? What should I check?	Debugging database seeding issues
ChatGPT 5.3 Plus	How to fix Prisma error when schema is updated but database is not reflecting changes?	Resolving migration sync problems
ChatGPT 5.3 Plus	Why is Vercel build stricter than local dev for TypeScript errors?	Understanding deployment issues
ChatGPT 5.3 Plus	How to fix this issue where Next.js dynamic route params are not recognized properly?	Fixing App Router typing issues
ChatGPT 5.3 Plus	Should I use params: { id: string } or Promise-based params in Next.js App Router?	Clarifying routing structure
ChatGPT 5.3 Plus	Why is my Next.js page breaking due to validator.ts type mismatch?	Debugging build-time type errors
ChatGPT 5.3 Plus	How to safely handle optional values like row.transportAllowance ?? 0 in calculations?	Improving defensive coding
ChatGPT 5.3 Plus	How to design a reusable payroll calculation function instead of duplicating logic?	Refactoring payroll logic
ChatGPT 5.3 Plus	How to calculate gross, deductions, and net pay step by step in code?	Structuring financial calculations
ChatGPT 5.3 Plus	Why are my Quebec tax values too high compared to expected output?	Validating payroll formulas
ChatGPT 5.3 Plus	How to adjust federal and Quebec tax using claim amounts and additional tax fields?	Refining tax calculation logic
ChatGPT 5.3 Plus	How to debug when EI or QPP is not calculating correctly?	Debugging deduction functions
ChatGPT 5.3 Plus	How to integrate YTD calculations into paystub generation?	Enhancing payroll reporting
ChatGPT 5.3 Plus	How to fetch all previous pay statements to calculate YTD values?	Backend data aggregation
ChatGPT 5.3 Plus	Why is my API route returning correct data but UI is not updating?	Debugging frontend state issues
ChatGPT 5.3 Plus	How to fix React Query not refetching after mutation?	Fixing async data updates

ChatGPT 5.3 Plus	Why is my modal not updating after submitting changes?	Debugging UI re-render issues
ChatGPT 5.3 Plus	How to structure admin pages so that navigation is consistent across sections?	Improving dashboard UX
ChatGPT 5.3 Plus	How to organize features like Leave, Payroll, and Timesheets in a clean UI?	Designing admin interface
ChatGPT 5.3 Plus	What is a better icon for generate paystub or payroll export?	Improving UI semantics
ChatGPT 5.3 Plus	How to keep spacing consistent between buttons in Mantine?	UI alignment fixes
ChatGPT 5.3 Plus	How to convert table data into downloadable CSV or Excel format?	Adding export functionality
ChatGPT 5.3 Plus	How to flatten nested JSON data before exporting to Excel?	Data transformation for reporting
ChatGPT 5.3 Plus	Why is my MultiSelect not showing options even though data exists?	Debugging component binding
ChatGPT 5.3 Plus	How to fix incorrect key mapping in list rendering in React?	Fixing UI rendering bugs
ChatGPT 5.3 Plus	How to fix useQuery returning undefined initially?	Handling async loading states
ChatGPT 5.3 Plus	How to structure Prisma include/select to optimize queries?	Improving database queries
ChatGPT 5.3 Plus	How to avoid over-fetching data in Prisma queries?	Performance optimization
ChatGPT 5.3 Plus	What is the best way to structure API routes in Next.js App Router?	Backend architecture design
ChatGPT 5.3 Plus	How to secure API routes using next-auth JWT tokens?	Implementing authentication
ChatGPT 5.3 Plus	Why is getToken returning null in my API route?	Debugging authentication issues
ChatGPT 5.3 Plus	How to explain my system as a SaaS solution in my report?	Academic writing support
ChatGPT 5.3 Plus	How to justify using PWA instead of native apps for my project?	Design decision explanation
ChatGPT 5.3 Plus	What should I include in evaluation techniques for my project?	Report writing guidance
ChatGPT 5.3 Plus	Would you be able to convert my schema (only the entities) to a mermaid code diagram?	Database Design Representation in the Report
ChatGPT 5.3 Plus	How to show my folder structure in the report? Can I use mermaid for this too?	Codebase Structure Representation in the Report
ChatGPT 5.3 Plus	How to convert my report into presentation slides with enough detail?	Presentation preparation
ChatGPT 5.3 Plus	How to improve readability of my report without making it too long?	Writing optimization

AI Model	AI Prompt	Context / Purpose / Value Addition
ChatGPT 5.3 Plus - Codex	Redesigning admin and staff UI across dashboard, tasks, payroll, and profile pages	Improved consistency and usability across multiple UI components
ChatGPT 5.3 Plus	Integrating AI features into job modal and appointment workflows	Enabled embedding AI into real workflows
ChatGPT 5.3 Plus	Refactoring application to centralize state and querying	Reduced duplication and improved maintainability
ChatGPT 5.3 Plus	Fixing task appointment detail issues	Improved stability of task detail functionality
ChatGPT 5.3 Plus	Improving login and authentication flow	Strengthened reliability of protected routes
ChatGPT 5.3 Plus	Refining PWA experience	Enhanced mobile usability
ChatGPT 5.3 Plus	Finalizing task screen	Improved UI and user flow
ChatGPT 5.3 Plus - Codex	Cleaning warnings	Improved code quality
ChatGPT 5.3 Plus	Implementing AI task assistant	Delivered key AI feature
ChatGPT 5.3 Plus	Documenting AI workflow	Improved clarity for reporting
ChatGPT 5.3 Plus - Codex	UI improvements	Enhanced visual quality
ChatGPT 5.3 Plus	Creating Job Edit Modal	Improved admin workflows
ChatGPT 5.3 Plus	Resolving merge conflicts	Reduced integration issues
ChatGPT 5.3 Plus - Codex	Fixing timezone issues	Ensured scheduling accuracy
ChatGPT 5.3 Plus - Codex	Stabilizing UI	Improved system reliability
ChatGPT 5.3 Plus	Cleaning codebase	Reduced technical debt
ChatGPT 5.3 Plus	Fixing build errors	Enabled successful deployment
ChatGPT 5.3 Plus - Codex	Improving PWA UX	Better mobile experience
ChatGPT 5.3 Plus - Codex	Ensuring UI consistency	Improved professionalism
ChatGPT 5.3 Plus - Codex	Fixing lint issues	Improved code consistency
ChatGPT 5.3 Plus	Updating README	Improved documentation

13. References

Prisma. (n.d.). Introduction to Prisma. Prisma Documentation. <https://www.prisma.io/docs>

Supabase. (n.d.). Supabase – The open-source Firebase alternative. <https://supabase.com/>

UploadThing. (n.d.). Next.js App Router setup. UploadThing Documentation. <https://docs.uploadthing.com/getting-started/appdir>

Resend. (n.d.). Introduction. Resend Documentation. <https://resend.com/docs/introduction>

OpenAI. (n.d.). Developer quickstart. OpenAI API Documentation. <https://developers.openai.com/api/docs/quickstart/>

Nettoyage Eco Vert. (2026, January 20). Home. Nettoyage Eco Vert. <https://nettoyageecovert.ca/>

Vercel. (2026, February 20). Getting started (App Router). Next.js Documentation. <https://nextjs.org/docs/app/getting-started>

Mantine. (2025, May 5). Version 8.0.0 (Changelog). Mantine.dev. <https://mantine.dev/changelog/8-0-0/>